

# SE 322 – Testing Project Report Template

## *Library Management System Testing Project*

**Submission Note:** Only one member from each group needs to submit the report together with the GitHub repository link. The project is worth 20% of the final grade.

### 1. Group Information

Fill in the names and student numbers of all group members. Each group should normally have 5–6 members.

| No. | Name Surname      | Student Number |
|-----|-------------------|----------------|
| 1   | Burcu Dumanlı     | 20230601025    |
| 2   | Ece Kabasakal     | 20230601043    |
| 3   | Beyza Barak       | 20220601007    |
| 4   | Alperen Demirezen | 20210601021    |
| 5   | Mert Koçyiğit     | 20210601039    |
| 6   | Zeynep Simay Dağ  | 20220601014    |

### 2. Project Description

In this project, our group worked on testing a Spring Boot based Library Management System REST API. The purpose of the project was to evaluate whether the application works correctly under different scenarios and to ensure that the system behaves reliably according to the given business rules.

The application is designed to manage basic library operations such as managing books, members, and borrowing records. During the testing process, we focused on validating important system behaviors including borrowing limits for different membership types, preventing duplicate borrow operations, checking member activity status, verifying available book copies, and handling return operations correctly.

The project included unit tests, integration tests, and API tests. For unit testing, we used JUnit and Mockito to test the business logic of classes individually. Integration testing was performed using PostgreSQL and TestContainers in order to verify database operations and repository behaviors in a realistic environment. In addition, API tests were implemented to test the application from an end-to-end perspective by sending HTTP requests and validating responses and status codes.

Throughout the project, GitHub Actions and the CI/CD pipeline were also used to automatically run tests and verify the build process. This helped us monitor the stability of the application and detect possible issues during development.

### 3. Work Distribution

The responsibilities in the project were shared among the group members based on the different testing stages and project requirements.

Burcu and Mert started the project by focusing mainly on the unit testing part. They worked on testing the core business logic of the application, including service-layer methods, validation scenarios, and exception handling. During this process, they used JUnit and Mockito to verify that individual classes and methods were functioning correctly and independently.

Ece and Beyza were responsible for the integration testing phase until May 15. Their work included testing repository operations with PostgreSQL and TestContainers, checking database interactions, validating JPA queries, and ensuring that entity relationships and persistence operations were working as expected in a real database environment.

Beyza also prepared and organized the project report, compiling the contributions of all group members and documenting the overall testing process and project results.

Simay and Alperen handled the final review and system testing stages until May 25. They focused on checking the overall stability of the application, reviewing API-level functionality, validating end-to-end behaviors, and monitoring the CI/CD pipeline. They also contributed to debugging, fixing failed test scenarios, and making sure that the project was ready for submission.

### 4. GitHub Repository and CI/CD Status

**GitHub Repository Link:**  [GitHub - burcudumanl/library\\_management\\_Group5](https://github.com/burcudumanl/library_management_Group5)

**CI/CD Pipeline Status:** ☒ All stages passed successfully ☐ Some stages failed

GitHub was used throughout the project to manage the repository and coordinate the work between group members. All updates, test implementations, fixes, and improvements were shared through the repository during the development process.

The project also included a CI/CD pipeline configured with GitHub Actions. The pipeline automatically executed the unit tests, integration/API tests, and build stages after updates to the repository. This helped us check whether the application and test structure were working properly and allowed us to identify possible issues during development. At the final stage of the project, all pipeline steps were completed successfully.

### 5. Additional Comments

During the project, one of the more challenging parts was testing different layers of the application together, especially the database integration and API testing processes. Setting up and running TestContainers with PostgreSQL sometimes required additional configuration and troubleshooting during development.

Overall, this project gave us valuable hands-on experience with software testing practices such as unit testing, integration testing, API testing, mocking, and CI/CD workflows. It also helped us improve our teamwork and collaboration skills while working with GitHub and version control throughout the project process.

If we had more time, we could further improve the project by adding more edge-case scenarios, security-related tests, and performance testing to make the system even more reliable and comprehensive.

